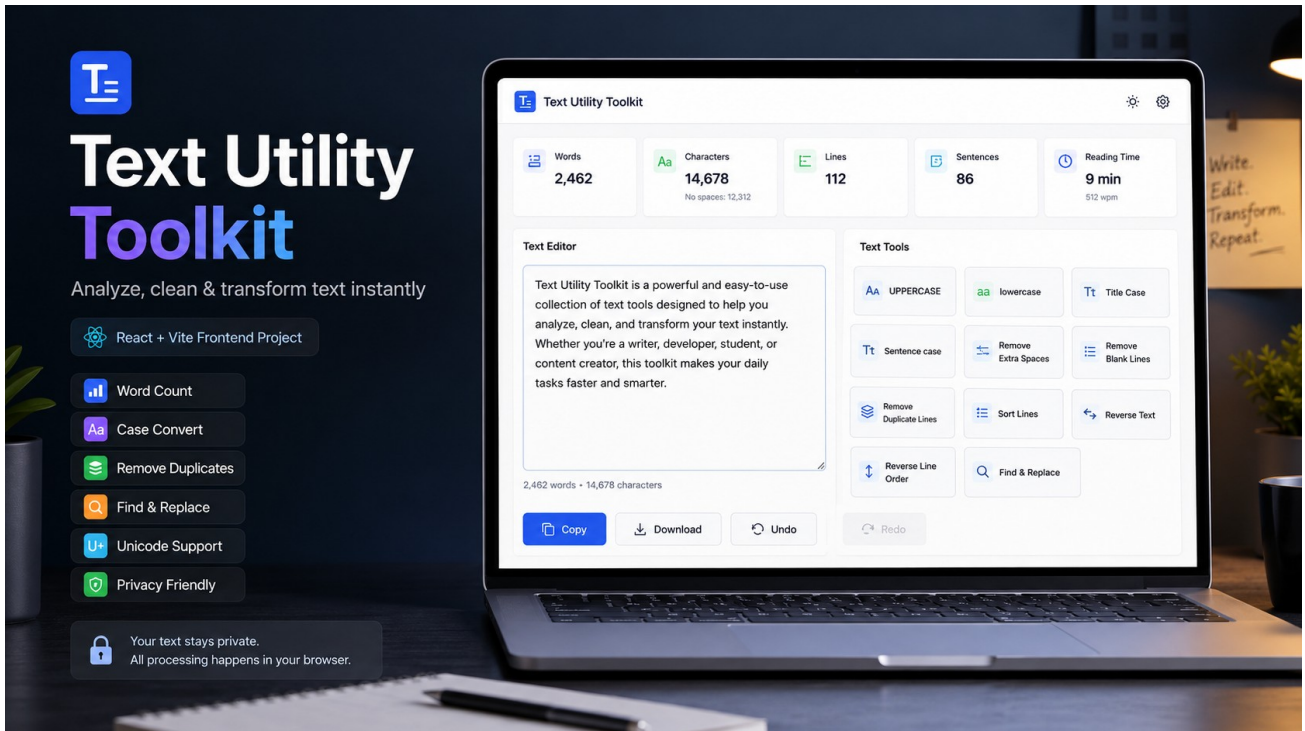


Text Utility Toolkit

A Privacy-First Browser Workspace for Text Analysis, Cleaning, Transformation & Export



Created by

Ann Noor Muhammad Mrinmoy

React 19 • Vite 7 • JavaScript • Browser APIs • Frontend Only

Portfolio • GitHub • University Demonstration • Viva

Document Map

How to use this report for learning, portfolio presentation, and viva preparation.

How this PDF is organized

The opening overview is intentionally in English for portfolio use. The deeper technical explanations use Bangla + English terminology so the implementation logic is easier to understand and explain.

Sections in this report

- 01 Executive Overview
- 02 Motivation & Idea Formation
- 03 Product Flow & Feature Map
- 04 Technology Stack
- 05 Architecture & Project Structure
- 06 React State & Application Orchestration
- 07 Text Statistics Logic
- 08 Cleaning & Transformation Logic
- 09 Find & Replace / Regex Safety
- 10 File Import, Copy & Download
- 11 Undo / Redo History Architecture
- 12 Privacy & localStorage Model
- 13 Unicode, Performance & Reliability
- 14 UI/UX, Responsive Design & Accessibility
- 15 Testing & Final Audit
- 16 How to Run, Build & Modify
- 17 Learning Outcomes
- 18 Limitations & Future Improvements
- 19 Portfolio & Viva Presentation Guide
- Appendix A: File-by-File Reference
- Appendix B: Viva Q&A

Source basis

This documentation is grounded in the actual final project source files, README.md, PROJECT_GUIDE.md, test suites, and VALIDATION.md. Where the report discusses “idea formation,” it describes the design rationale that logically leads to this product—not an unverifiable personal memory.

Executive Overview

A concise English summary suitable for portfolio readers and project evaluators.

Project Overview

Text Utility Toolkit is a privacy-first, frontend-only productivity application that combines common text-processing tasks inside one focused browser workspace. Users can write or paste text, view live statistics, clean and transform content, search and replace text, import local files, undo or redo explicit transformations, and export processed text without using a backend or database.

The application follows a simple product workflow: Write / Paste → Analyze → Clean → Transform → Export. The core design goal is to feel like a professional text workspace rather than a collection of disconnected utility buttons.

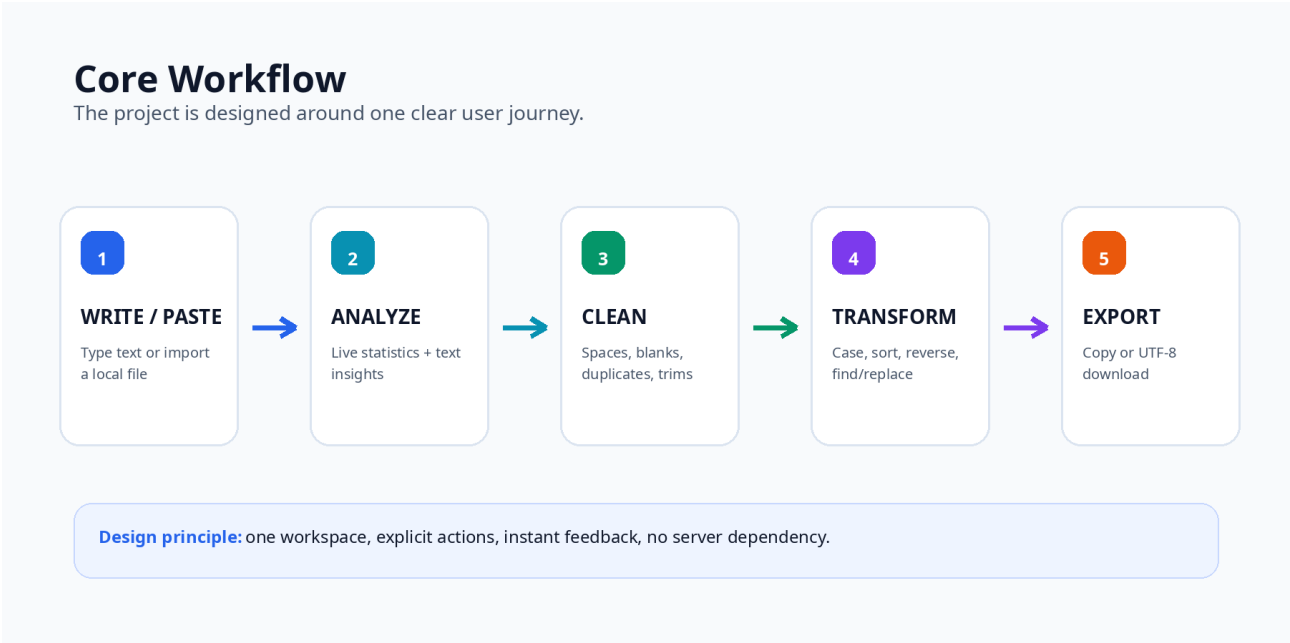


Figure 1 — The central user workflow implemented by the application.

Key Project Facts

Item	Implementation
Project type	Frontend-only React web application
Version	1.0.0
Primary language	JavaScript (ES modules)
UI framework	React 19.1.1
Build tool	Vite 7.1.3
Icons	Lucide React 0.541.0
Runtime requirement	Node.js 20+
Backend / Database	None
Processing model	Local browser processing

Default text persistence	Disabled; draft storage is opt-in
Audit command	npm run test:audit

What the Application Can Do

- Live word, character, line, sentence, paragraph, reading-time and unique-word statistics.
- UPPERCASE, lowercase, Title Case, Sentence case and Toggle Case.
- Remove extra spaces, trim text/lines, remove blank lines and normalize line endings.
- Remove duplicate lines with case and whitespace comparison options.
- Sort lines alphabetically or by length; reverse characters, line order or word order.
- Find & Replace with literal-search safety, case sensitivity and optional Regex mode.
- Word frequency analysis with a local stopword filter and a 150 ms debounce.
- Import .txt/.md/.csv files locally, copy to clipboard, and download UTF-8 text.
- Bounded Undo/Redo for explicit transformations, light/dark theme, responsive UI and accessibility support.

Portfolio positioning

This is not “just a word counter.” Its portfolio value comes from combining data processing, state management, Browser APIs, Unicode safety, undo/redo architecture, privacy design, responsive UI, accessibility, and automated audit tests in one coherent application.

Motivation & Idea Formation

Why this project exists and how the concept can be logically derived from a real user problem.

The Problem Behind the Project

Text-related micro-tasks are extremely common: counting words, cleaning copied notes, removing duplicates from a list, converting case, sorting lines, finding repeated text, and exporting the result. Normally, a user may jump between multiple websites or manually edit the text. That creates friction, inconsistency, and a privacy concern when the pasted content is sensitive.

বাংলায় সহজভাবে

ভাবো তুমি class note, assignment text, copied dataset, code/log, বা content নিয়ে কাজ করছ। একবার word count দরকার, পরে duplicate line remove, তারপর sort, তারপর find & replace। প্রতিটা কাজের জন্য আলাদা website use করা অপ্ৰয়োজনীয়। এখান থেকেই “সব text utility এক workspace-এ” idea তৈরি হয়।

Design Rationale — From Problem to Product

1. Identify repeated text-processing tasks that can be solved with deterministic JavaScript string/array logic.
2. Group those tools into a single workflow instead of a wall of unrelated buttons.
3. Keep processing inside the browser so no server is required and private text stays local.
4. Use React because the editor, statistics, settings, dialogs, search state and tool options all change interactively.
5. Add engineering depth: Unicode handling, file validation, Undo/Redo, responsive behavior, accessibility and tests.
6. Document the project so it can be explained clearly in a university viva and presented professionally in a portfolio.

Project Motive

Motive	Why it matters
Practical utility	Useful for students, writers, developers, content creators and data-cleaning work.
Privacy	Pasted text can contain notes, code, personal content or datasets; browser-local processing reduces exposure.
Learning React	Provides meaningful state, derived values, components, hooks and user interactions.
Algorithm practice	Set, Map, RegExp, sorting, tokenization, stacks and string transformations are used in a real UI.
Portfolio depth	Demonstrates more than visual design: architecture, testing, accessibility, APIs and performance choices.
Viva readiness	Pure utility functions and clear modules make the logic easy to explain and defend.

Product Flow & Feature Map

What happens from the user's point of view, before we look at code.

User Journey

The interface is centered on the editor. Statistics react to the current text, tools explicitly transform that text, history provides recovery, and copy/download provides the final output. This keeps the product understandable even though many utilities are available.

1) Write or Paste

The user types directly in a plain `<textarea>`, pastes content, loads sample text, or imports a supported local text file. A plain textarea was intentionally used because the project does not need complex rich-text formatting.

2) Analyze

`calculateStats()` computes lightweight live metrics. Word-frequency analysis is slightly heavier, so it uses a debounced text value and recomputes after about 150 ms instead of on every immediate keystroke.

3) Clean and Transform

Tools are grouped into categories such as Case, Cleanup, Lines, Transform, Find & Replace, and Advanced. Each explicit transformation calculates a new text value, records the previous state in history, then updates the UI.

4) Export or Recover

Users can Undo/Redo transformation states, copy current text with the Clipboard API, or download a UTF-8 text file using a Blob and temporary object URL.

Core Feature Groups

Group	Representative tools	Core technique
Statistics	Words, characters, lines, sentences, reading time	Tokenization, regex, Set, derived state
Case	Upper, lower, title, sentence, toggle	Locale-aware casing + Unicode iteration
Cleanup	Spaces, trim, blank lines, normalize	Regex + line-preserving transformations
Lines	Duplicates, A→Z, Z→A, length sort	Set + localeCompare()
Transform	Reverse characters/lines/words	Intl.Segmenter / Array.from()
Find & Replace	First/all/remove, regex option	Escaped RegExp + error handling
Analysis	Top words, unique words	Map + Set + debounce
Files	Import, drag/drop, download	File API + Blob API
Safety	Undo/Redo, confirmations	Reducer + bounded history

Technology Stack

Which languages, libraries, browser APIs and tools are used—and why.

Languages and Frameworks

Technology	Actual use in this project	Why it was selected
HTML	<code>`index.html`</code> provides the root mount point.	Minimal shell for a single-page React app.
CSS	<code>globals.css</code> , <code>app.css</code> , <code>responsive.css</code>	Complete visual system, themes, layout, focus and breakpoints.
JavaScript	All UI and text-processing logic.	Native browser text APIs and ES module architecture.
React 19	Components, state, hooks, rendering.	Interactive editor + many synchronized UI states.
Vite 7	Development server and production build.	Fast React workflow with minimal configuration.
Lucide React	Consistent interface icons.	Accessible, lightweight visual language.

Browser APIs Used

- File API / ``File.text()`` — reads user-selected text files locally.
- Clipboard API — copies current editor text when the browser permits it.
- Blob + ``URL.createObjectURL()`` — creates downloadable UTF-8 text without a server.
- localStorage — persists theme/settings and only stores draft text when the user explicitly enables auto-save.
- ``Intl.Segmenter`` — improves word and grapheme segmentation when supported.
- ``RegExp``, ``Set``, ``Map``, ``Array.from()`` and ``localeCompare()`` — core language tools used for processing.

কেন **backend/database** লাগেনি?

এই project-এর main data হলো temporary editor text। কোনো login, shared account, server record, payment, multi-user collaboration বা permanent database record দরকার নেই। তাই backend যোগ করলে complexity বাড়ত কিন্তু core problem solve করতে value যোগ করত না।

Development / Quality Tools

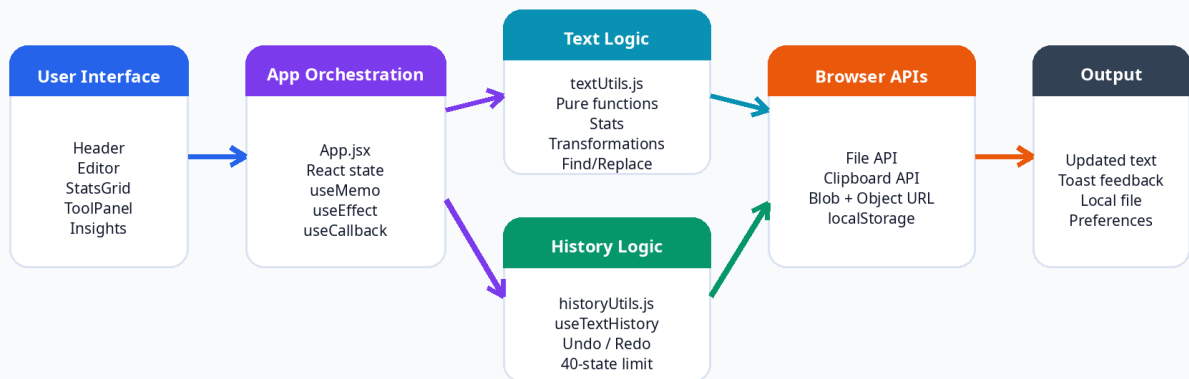
The repository also includes Node-based audit tests. ``npm run test:audit`` runs ``tests/text-utils.test.mjs`` and ``tests/project-audit.test.mjs``, which verify logic, history behavior, file validation, privacy-sensitive code patterns, responsive CSS coverage and other project invariants.

Architecture & Project Structure

How the codebase is separated so the logic remains testable, explainable and maintainable.

Application Architecture

Separation of UI, state/history, pure text logic, and browser APIs.



No backend or database: all text processing stays inside the browser.

Figure 2 — Source-level architecture of the final project.

Architectural Principle: UI $\neq$ Text Logic

The strongest structural decision is that text-processing functions do not live inside button components. Most transformations are pure functions inside `src/utils/textUtils.js`. React components focus on user interaction and rendering; utility modules focus on deterministic logic.

বাংলায় মূল ধারণা

Button-এর কাজ button দেখানো এবং click event পাঠানো। Duplicate remove করার algorithm button-এর ভিতরে লিখিনি। Algorithm `textUtils.js`-এ আলাদা pure function হিসেবে আছে। ফলে একই function test করা, debug করা, future-এ reuse করা এবং viva-তে explain করা সহজ।

Project Structure

```

src/
  components/    → UI sections and dialogs
  data/          → sample text and stopwords
  hooks/         → debouncing and text-history hook
  styles/        → global theme, app layout, responsive CSS
  utils/         → text, file, history and storage logic
  App.jsx        → application orchestration
  main.jsx       → React entry point
  tests/         → utility and project audit tests
  README.md      → public project documentation
  PROJECT_GUIDE.md → learning + viva guide
  VALIDATION.md  → final audit status
  
```


Important Files and Responsibilities

File	Responsibility
src/App.jsx	Owns top-level application state, derived calculations and action orchestration.
src/utls/textUtils.js	Pure statistics, cleanup, transform, sort, reverse and Find/Replace functions.
src/utls/historyUtils.js	Pure reducer that implements bounded Undo/Redo behavior.
src/hooks/useTextHistory.js	Wraps the history reducer in a React-friendly API.
src/utls/fileUtils.js	File validation/read, Clipboard copy and Blob download.
src/utls/storage.js	Safe localStorage read/write/remove with fallbacks.
src/components/ToolPanel.jsx	Categorized tool interface and Find/Replace controls.
src/components/Editor.jsx	Textarea, action controls, drag/drop and editor settings.
src/components/TextInsights.jsx	Unique words and word-frequency visualization.
src/styles/*.css	Theme tokens, application layout, breakpoints and focus behavior.
tests/*.mjs	Logic and architecture regression audits.

React State & Application Orchestration

How App.jsx connects the editor, settings, tools, history and feedback.

Top-Level State

`App.jsx` is the coordinator. It stores settings and UI options, calls pure utility functions, dispatches transformations to history, manages file import decisions, and passes focused props to child components.

State / derived value	Purpose
settings	Font size, monospace, word wrap and optional draft auto-save.
theme	Light or dark mode.
history.text	The current editor text managed through `useTextHistory`.
duplicateOptions	Case-sensitive and surrounding-space comparison behavior.
sortCaseSensitive	Controls alphabetic sorting sensitivity.
findState	Find text, replacement text, case option and regex option.
ignoreCommonWords	Controls stopword filtering for word frequency only.
toast / dialogs	Feedback and safe destructive actions/import choices.
stats / frequency / findMeta	Derived values computed with memoization/debouncing.

Hooks and Why They Matter

Hook	How it is used
useState	Interactive settings, options, dialog visibility, theme and toast state.
useMemo	Caches statistics, word-frequency result and match metadata when inputs have not changed.
useEffect	Synchronizes theme/settings/draft storage and keyboard listeners.
useCallback	Keeps action handlers stable and centralizes behavior.
useRef	Keeps file input and toast timer references.
useReducer	Used inside the history hook for predictable state transitions.

The applyTool() Pattern

Most tool buttons do not directly edit the textarea. They send a tool identifier to `applyTool()`. The function reads current text, selects the corresponding utility function, calculates the new value, records it as a transformation, and sends a toast message.

Simplified flow from App.jsx

```
case 'duplicates': {
  const result = removeDuplicateLines(current, duplicateOptions);
  next = result.text;
  message = `${result.removed} duplicate lines removed`;
  break;
}

history.applyTransformation(next);
notify(message);
```

কেন এই **pattern** ভালো?

Tool action-এর orchestration এক জায়গায় থাকে। UI button, processing logic, history update এবং feedback-এর responsibility আলাদা থাকে। নতুন tool add করলেও একই predictable flow follow করা যায়।

Text Statistics Logic

How the application converts raw text into useful live metrics.

Word Count

The project first attempts `Intl.Segmenter` with word granularity. If the browser does not support it or segmentation fails, a Unicode-aware regular expression is used as a fallback. Empty text returns zero.

```
const segmenter = new Intl.Segmenter(undefined, { granularity: 'word' });
return Array.from(segmenter.segment(trimmed))
  .filter((part) => part.isWordLike)
  .map((part) => part.segment);
```

বাংলায় বুঝে রাখো

Simple `text.split(" ")` ব্যবহার করলে multiple spaces, tabs, newlines এবং multilingual text-এ ভুল count হতে পারে। তাই projectটা better tokenization strategy ব্যবহার করেছে।

Character Metrics

Metric	Implementation meaning
Characters	<code>Array.from(text).length</code> — code-point-aware count.
Characters without spaces	Only literal spaces (<code>` `</code>) are removed before counting.
Characters without whitespace	Spaces, tabs, newlines and JavaScript <code>`\s`</code> whitespace are removed.

Line and Sentence Count

Line endings are normalized from CRLF/CR to LF (```) so Windows and Unix text behave consistently. Empty text is explicitly treated as 0 lines. Sentence count is heuristic: runs of ``., !`` or ``?`` at reasonable endings are counted as one sentence boundary, so ``...`` and ``?!`` do not become absurdly large counts.

Reading Time and Insights

- Reading time = word count ÷ 200 words per minute.
- Unique words = size of a Set created from lowercase normalized tokens.
- Average word length = total token character length ÷ number of tokens.
- Longest word = the token with the largest Unicode-aware character length.
- Paragraphs = non-empty blocks separated by blank lines.

Limitation to explain in viva

Sentence detection and multilingual tokenization are practical heuristics, not a full Natural Language Processing parser. The project documents this instead of claiming perfect linguistic accuracy.

Cleaning & Transformation Logic

The algorithms behind case conversion, cleanup, duplicates, sorting and reversal.

Case Conversion

Tool	Logic
UPPERCASE	<code>`toLocaleUpperCase()`</code> preserves non-case characters and line structure.
lowercase	<code>`toLocaleLowerCase()`</code> .
Title Case	Unicode letter regex finds word-like runs, lowercases them, then capitalizes the first character.
Sentence case	Lowercases text, then capitalizes the next letter after start / <code>` . ! ? `</code> / newline.
Toggle Case	Iterates characters and inverts only characters that actually have case.

Cleanup Tools

A key requirement is preserving line structure. For example, Remove Extra Spaces normalizes line endings, processes each line separately, collapses repeated spaces/tabs inside that line, and rejoins lines with ```

```. It does not flatten the entire document.

### Remove Extra Spaces — line-preserving approach

```
return normalizeLineBreaks(text)
 .split('\n')
 .map((line) => line.replace(/[\t]{2,}/g, ' '))
 .join('\n');
```

# Duplicate Line Removal — Why Set is Used

## Example: Remove Duplicate Lines

A concrete walkthrough of one transformation from click to result.

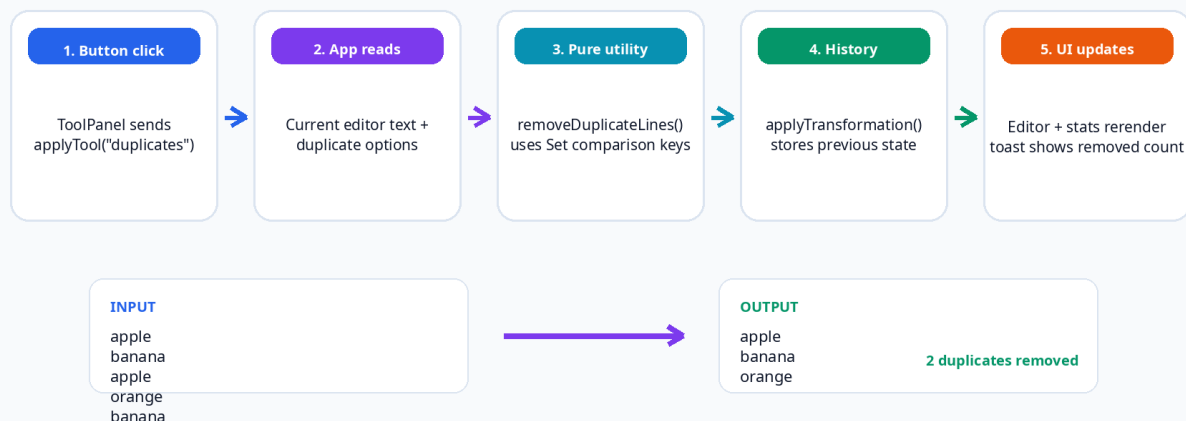


Figure 3 — Click-to-result flow for duplicate line removal.

The algorithm scans lines from top to bottom. A comparison key is optionally trimmed and optionally lowercased. If the key has not been seen, the original line is preserved. This means first occurrence order stays stable, and “ignore surrounding spaces” does not unexpectedly trim the line that is kept.

```
const seen = new Set();
const kept = [];
for (const line of lines) {
 let key = ignoreSurroundingSpaces ? line.trim() : line;
 if (!caseSensitive) key = key.toLocaleLowerCase();
 if (!seen.has(key)) { seen.add(key); kept.push(line); }
}
```

## Sorting

Alphabetic sorting works on a copied line array and uses `localeCompare()` with numeric comparison. This avoids silently trimming, deduplicating or lowercasing the actual line content. Separate modes sort shortest→longest and longest→shortest.

## Unicode-Aware Reverse

Reversing with `split("")` can break surrogate pairs and emoji. The project prefers `Intl.Segmenter` with grapheme granularity and falls back to `Array.from()`, which is safer for code points.

```
const segmenter = new Intl.Segmenter(undefined, { granularity: 'grapheme' });
return Array.from(segmenter.segment(text), p => p.segment)
 .reverse()
 .join('');
```

# Find & Replace / Regex Safety

How the project supports powerful searching without making normal text search dangerous.

## Plain Text vs Regex Mode

The application intentionally separates normal search from regular-expression search. In normal mode, regex metacharacters are escaped. Therefore searching for `.` finds literal dots instead of every character. Regex interpretation happens only when the user explicitly enables the Advanced Regex option.

```
export function escapeRegex(value) {
 return value.replace(/[*+?^$()\[\]\\" data-bbox="94 308 906 354"/>

```

## Available Operations

- Replace First
- Replace All
- Remove All Matches
- Case-sensitive option
- Optional Regex mode
- Live match count
- Invalid-regex validation without application crash

## Replacement Safety Detail

In plain-text mode, replacement is passed through a callback so JavaScript replacement tokens such as `\$&` are treated as literal replacement text. In Regex mode, normal JavaScript replacement-token behavior is intentionally preserved because the user explicitly chose regex semantics.

### Viva point

The important engineering choice is not “using regex.” It is correctly escaping normal user input and wrapping optional regex mode in `try/catch` so malformed patterns do not crash the application.

# File Import, Copy & Download

How local Browser APIs replace the need for upload endpoints and server-side file generation.

## Import Flow

7. User selects or drops a `.txt`, `.md` or `.csv` file.
8. `validateTextFile()` checks extension, size and useful MIME information.
9. Maximum size is 5 MB to reduce accidental browser freezes.
10. `file.text()` reads the content locally; no upload request is created.
11. If editor text already exists, the UI asks Replace / Append / Cancel instead of destroying current work silently.

```
export async function readTextFile(file) {
 const validation = validateTextFile(file);
 if (validation) throw new Error(validation);
 return file.text();
}
```

## Clipboard Copy

The preferred path is `navigator.clipboard.writeText(text)`. If the API is unavailable, the project creates a temporary hidden textarea, selects its content, attempts the older copy command, then removes the helper element.

## Download Without a Server

The project creates a UTF-8 text Blob, generates a temporary object URL, attaches it to a temporary anchor element, triggers the browser download, removes the anchor and revokes the URL.

```
const blob = new Blob([text], { type: 'text/plain;charset=utf-8' });
const url = URL.createObjectURL(blob);
anchor.href = url;
anchor.download = 'processed-text.txt';
anchor.click();
URL.revokeObjectURL(url);
```

### বাংলায় সহজভাবে

Server-এ file পাঠিয়ে আবার download করার দরকার নেই। Browser নিজেই memory-তে text file বানিয়ে user-কে download করতে পারে। এটিই frontend-only architecture-এর practical advantage.



# Undo / Redo History Architecture

Why Undo/Redo is more than two buttons—and how the reducer keeps it predictable.

## History State Model

```
{
 current: "current editor text",
 undo: [previousTransformationStates],
 redo: [undoneTransformationStates]
}
```

The project uses a pure reducer in `historyUtils.js`. Explicit transformations push the previous text into the Undo stack. Undo moves the current value into Redo and restores the newest Undo value. Redo performs the opposite transition.

## Why Only Explicit Transformations Are Recorded

Saving every keystroke as a full copy of a large text document can consume substantial memory. Manual typing changes the current text and clears a stale redo branch, but it is not continuously pushed as transformation history. This design focuses history on tool operations—the actions most likely to need safe reversal.

## Bounded Stack

`'HISTORY_LIMIT = 40'`. When a stack becomes larger than the limit, older states are dropped. This prevents unbounded memory growth while keeping enough recovery depth for normal use.

## Branch Invalidation

If the user Undoes something and then performs a new transformation—or manually types new content—the previous Redo branch is cleared. This matches standard editor behavior because the old future state no longer belongs to the new editing branch.

### Viva example

A → Uppercase → Remove Spaces → Undo → New Sort. After the new Sort, Redo should not return to the old Remove Spaces future. The reducer explicitly clears that incompatible branch.

# Privacy & localStorage Model

What “privacy-first” means in actual source code, not just marketing copy.

## Privacy Rules Implemented

- No backend code is present.
- No database is used.
- No external text-processing API is called.
- No `fetch()`, `Axios`, `XMLHttpRequest` or `WebSocket` text path exists in `src/` according to the project audit.
- Imported text files are read locally.
- User editor text is not written to `localStorage` by default.
- Optional draft persistence is explicitly opt-in and can be cleared.

## localStorage Keys

| Key                                  | Purpose                                       | Contains user text? |
|--------------------------------------|-----------------------------------------------|---------------------|
| <code>textUtility.theme</code>       | Light/dark preference                         | No                  |
| <code>textUtility.settings</code>    | Editor preferences including auto-save toggle | No                  |
| <code>textUtility.dataVersion</code> | Preference schema/version marker              | No                  |
| <code>textUtility.draft</code>       | Draft content only when auto-save is enabled  | Yes — opt-in only   |

## Safe Storage Wrapper

All direct `localStorage` access is centralized in `storage.js`. Read/write operations are wrapped in `try/catch`, so blocked storage or malformed JSON does not crash the interface.

```
export function safeRead(key, fallback) {
 try {
 const raw = localStorage.getItem(key);
 return raw == null ? fallback : JSON.parse(raw);
 } catch {
 return fallback;
 }
}
```

### Privacy claim you can safely make

“All text processing happens locally in the browser. The project has no backend/database, and editor text is not permanently stored by default.” This statement matches the actual source and audit.

# Unicode, Performance & Reliability

Engineering details that make the toolkit safer than naive string-manipulation demos.

## Unicode Handling

The code avoids ASCII-only assumptions where practical. It uses Unicode property escapes, locale-aware casing, `Array.from()` and `Intl.Segmenter`. Tests include Bangla, mixed English/Bangla, emoji, Unicode digits and grapheme-like emoji sequences.

| Problem         | Naive approach                     | Project approach                                                        |
|-----------------|------------------------------------|-------------------------------------------------------------------------|
| Word tokens     | <code>`split(" ")`</code>          | <code>Intl.Segmenter</code> + Unicode-aware regex fallback              |
| Character count | <code>`text.length`</code> only    | <code>Array.from(text).length</code>                                    |
| Reverse emoji   | <code>`split("").reverse()`</code> | <code>Grapheme Segmenter</code> , then <code>Array.from</code> fallback |
| Remove numbers  | <code>`[0-9]`</code> only          | Unicode <code>`\p{N}`</code> with fallback                              |
| Punctuation     | ASCII symbol list                  | Unicode <code>`\p{P}`</code> + <code>`\p{S}`</code> with fallback       |

## Performance Choices

- Basic statistics use `useMemo` so they recalculate only when editor text changes.
- Word frequency uses a custom `useDebounceValue` with a 150 ms delay.
- Undo/Redo is bounded to 40 transformation states.
- Imported files are capped at 5 MB.
- The audit includes a 10,000-line large-text smoke test and requires completion within 5 seconds.

## Reliability Patterns

- Pure text functions make outputs deterministic and testable.
- Regex parsing is guarded with error handling.
- Storage parsing uses fallback values.
- Destructive Clear uses confirmation.
- Import into non-empty editor requires Replace/Append/Cancel choice.
- Toast feedback avoids disruptive browser `alert()` dialogs.

# UI/UX, Responsive Design & Accessibility

How the interface supports practical desktop and mobile use without sacrificing clarity.

## UI Design Strategy

The interface prioritizes the text editor and live statistics. Tool actions are grouped by category rather than displayed as a giant button wall. A clear hierarchy—intro → stats → editor/tools → insights → privacy note—supports the main workflow.

## Responsive Breakpoints

| Target  | Relevant CSS behavior                                                   |
|---------|-------------------------------------------------------------------------|
| 1440 px | Default desktop: statistics grid and two-column editor/tools workspace. |
| 1024 px | Rules under max-width 1180px adjust stats and column proportions.       |
| 768 px  | Under 920px the workspace switches to a single column.                  |
| 430 px  | Under 680px tool tabs become a compact grid; editor/actions reflow.     |
| 375 px  | Under 420px tool grid becomes one column and insight metrics stack.     |

## Accessibility Features

- Semantic sections/headings and explicit input labels.
- ARIA labels on icon-only buttons.
- Visible keyboard focus styles and readable contrast.
- Keyboard-operable dialogs with Escape handling, focus trapping and focus restoration.
- Tool-category tab navigation with Arrow Left/Right, Home and End.
- `prefers-reduced-motion` support for users who reduce animations.

## Keyboard Shortcuts

| Shortcut               | Action / behavior                                                      |
|------------------------|------------------------------------------------------------------------|
| Alt + U                | UPPERCASE                                                              |
| Alt + L                | lowercase                                                              |
| Alt + D                | Remove Duplicate Lines                                                 |
| Ctrl/Cmd + Y           | Redo                                                                   |
| Ctrl/Cmd + Z / Shift+Z | History shortcuts where appropriate; textarea native undo is respected |

# Testing & Final Audit

What was actually verified, and which environment limitation remains.

## Automated Audit Coverage

- Empty statistics return zeros without NaN/undefined/Infinity.
- Word counting across spacing, tabs/newlines, Bangla, mixed language and emoji-containing text.
- Character metrics, CRLF line counting and sentence punctuation runs.
- All case conversion and cleanup tools.
- Duplicate modes, sorting preservation and Unicode reversal.
- Literal Find & Replace safety for `.` `^` `?` `[` plus Regex success/error cases.
- Word frequency, stopwords, unique words and Unicode number removal.
- Undo/Redo branch behavior and the 40-state limit.
- File extension/MIME/size validation.
- Large-text smoke test using 10,000 lines.
- Project-level privacy scan and source import resolution checks.
- Responsive breakpoint and accessibility-related code checks.

### Audit result

`npm run test:audit` passed in the final source audit. JavaScript/JSX syntax parsing also passed.

## Production Build Status — Important Honesty

During the final sandbox audit, `npm install` could not download packages because the environment failed to resolve `registry.npmjs.org` with an `EAI\_AGAIN` DNS error. Therefore Vite was unavailable in that sandbox and the actual `npm run build` / `npm run dev` execution could not be independently confirmed there.

### What this means

Source-level tests, privacy scans and syntax checks passed. However, this report does not falsely claim that a Vite production build passed inside the restricted sandbox. Run the normal commands on an internet-connected machine to complete that final environment-specific verification.

## Representative Test Cases

| Test                    | Expected behavior                                        |
|-------------------------|----------------------------------------------------------|
| Empty text              | Words 0, Characters 0, Lines 0, Sentences 0              |
| “আমি ভালো আছি”          | 3 words in the audit environment                         |
| `a`<br>b`               | 2 lines after newline normalization                      |
| `Wait... What?!`        | 2 estimated sentences                                    |
| Apple / apple duplicate | Both remain case-sensitive; one remains case-insensitive |

|              |                                                 |
|--------------|-------------------------------------------------|
| Search ``    | Matches literal dot in normal mode              |
| Reverse `😊a` | `a😊`                                            |
| 10,000 lines | Stats and transforms complete under audit limit |

## Recommended Manual Verification Before Release

- Open the app on desktop and a narrow mobile viewport.
- Type, paste, import, append and replace text manually.
- Perform several transformations and verify Undo/Redo visually.
- Check keyboard focus in dialogs and test Escape close behavior.
- Download Bangla/mixed text and reopen the file to confirm UTF-8 content.
- Run the production build on an internet-connected machine before deployment.

# How to Run, Build & Modify

A practical Windows-friendly guide for running the project locally and extending it.

## Requirements

- Node.js 20 or newer (the package declares `node >=20`).
- npm, included with normal Node.js installation.
- A modern browser such as Chrome, Edge or Firefox.
- VS Code is recommended but not required.

## Run Locally

```
cd path\to\text-utility-toolkit
npm install
npm run test:audit
npm run dev
```

Vite prints a local URL in the terminal. Open that URL in the browser. Keep the terminal running while using the development server.

## Production Build

```
npm run build
npm run preview
```

A successful Vite build normally creates a `dist/` folder. `npm run preview` serves that production output locally for verification.

## How to Add a New Text Tool

12. Create a pure function in `src/utils/textUtils.js`.
13. Add test cases in `tests/text-utils.test.mjs`.
14. Add a new action case in `applyTool()` inside `App.jsx`.
15. Add the UI button in the appropriate category in `ToolPanel.jsx`.
16. Provide clear user feedback through the existing toast system.
17. Run `npm run test:audit`, then build and manually test desktop/mobile layouts.

## Common Problems

| Problem                    | Fix                                                                   |
|----------------------------|-----------------------------------------------------------------------|
| `npm` not found            | Install Node.js and reopen terminal/VS Code.                          |
| Invalid regex              | Disable Regex mode or correct the expression.                         |
| Clipboard denied           | Use localhost/HTTPS and a modern browser; fallback copy is attempted. |
| File too large             | Use a text file under 5 MB.                                           |
| Theme preference corrupted | Clear site localStorage; safe fallbacks also protect loading.         |

|                         |                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------|
| Build error after edits | Check exact terminal line, imports, JSX syntax and dependencies; run audit tests first. |
|-------------------------|-----------------------------------------------------------------------------------------|

## Before Pushing to GitHub

- Run ``npm run test:audit``.
- Run ``npm run build`` and confirm the ``dist/`` output.
- Remove accidental console/debug code or temporary files.
- Check README run commands and screenshots/thumbnail.
- Verify the privacy note still matches the source behavior.



# Learning Outcomes

What this project teaches beyond simply building the visible interface.

## What You Learn Technically

| Skill                  | What you should be able to explain                                                                        |
|------------------------|-----------------------------------------------------------------------------------------------------------|
| React component design | কোন অংশ reusable component হবে আর কোন logic utility module-এ থাকবে—এই separation বোঝা।                    |
| State management       | Editor state, options, modal visibility, settings, derived stats এবং history state কীভাবে coordinate হয়। |
| Pure functions         | UI থেকে algorithm আলাদা রাখলে testing/debugging সহজ হয়।                                                  |
| Data structures        | Duplicate remove-এ Set, frequency-তে Map, unique words-এ Set, Undo/Redo-তে stack-like arrays।             |
| Regular expressions    | Text cleaning, tokenization, literal escaping, optional advanced regex handling।                          |
| Browser APIs           | File, Clipboard, Blob, Object URL, localStorage practically ব্যবহার করা।                                  |
| Unicode thinking       | Bangla/emoji support করতে ASCII-only assumption কেন dangerous সেটা বোঝা।                                  |
| Performance            | useMemo, debounce, file-size limits, bounded history দিয়ে unnecessary work/memory control।               |
| Responsive design      | Desktop layout simply shrink না করে mobile-এ structure reflow করা।                                        |
| Accessibility          | ARIA, focus, keyboard navigation, reduced motion—professional frontend-এর অংশ।                            |
| Testing                | Expected behavior আগে define করে utility functions এবং project invariants audit করা।                      |
| Privacy engineering    | “No backend” শুধু feature নয়; storage/network behavior source level-এ verify করা।                        |

সবচেয়ে গুরুত্বপূর্ণ শেখা

### 1 — UI আর logic আলাদা করা

React project সুন্দর দেখানোই শেষ কথা না। Data transformation pure function-এ রাখলে codebase professional হয় এবং test করা যায়।

## 2 — Edge case চিন্তা করা

Empty text, repeated punctuation, CRLF, duplicate capitalization, emoji, invalid regex, oversized file—এসব handle করাই demo থেকে real project-এর পার্থক্য তৈরি করে।

## 3 — Honest limitations

Sentence detection perfect না, multilingual segmentation browser ভেদে vary করতে পারে—এগুলো clearly document করা engineering maturity দেখায়।

# How These Skills Become Portfolio Evidence

- Show the source separation between components and pure utilities.
- Mention exact data structures: Set for duplicates, Map for frequency, reducer stacks for history.
- Demonstrate a real edge case such as literal-dot search or Unicode emoji reversal.
- Show the audit tests instead of only claiming that the app “works”.
- Explain why no backend/database is the correct architecture for this scope.

# Limitations & Future Improvements

What the current version deliberately does not overclaim, and where the project can evolve next.

## Current Limitations

- Sentence counting is heuristic, not full language-aware NLP.
- Title Case and Sentence case are general transformations, not AP/Chicago editorial rules.
- Word tokenization can vary by browser/language.
- Clipboard permissions depend on browser/security context.
- Extremely large text can still consume memory and CPU despite safeguards.
- Best grapheme handling depends on `Intl.Segmenter` support.
- No cloud synchronization or multi-device history exists by design.
- No visual before/after diff is implemented.

## Strong Future Improvements

| Improvement                      | Value                                                                        |
|----------------------------------|------------------------------------------------------------------------------|
| PWA / offline installation       | Turn the privacy-first utility into an installable desktop/mobile-like tool. |
| Before/After diff                | Make destructive transformations easier to inspect visually.                 |
| Custom transformation pipelines  | Users can build repeatable cleanup sequences.                                |
| Favorite tools                   | Pin frequently used actions for faster workflow.                             |
| Editable stopwords               | Better content-analysis customization.                                       |
| More language-aware tokenization | Improve metrics for additional scripts/languages.                            |
| Browser E2E tests                | Automate real click/file/dialog/accessibility flows.                         |
| Export presets                   | Controlled .txt/.md naming and newline options.                              |

### Scope discipline

A backend should only be added if future requirements genuinely need accounts, cloud sync, collaboration or server-side storage. Adding a database only to make the stack look “bigger” would weaken the architecture rather than improve it.

# Portfolio & Viva Presentation Guide

How to explain the project clearly without sounding like you only assembled UI components.

## 60-Second Portfolio Explanation

### Suggested explanation

"Text Utility Toolkit is a frontend-only React application that combines common text analysis and cleanup tasks in one privacy-first workspace. I separated the text-processing algorithms into pure JavaScript utilities and used React for UI state and orchestration. It includes Unicode-aware word/character handling, duplicate removal with Set, locale-aware sorting, safe Find & Replace with optional regex, local file import, Blob download, and a bounded Undo/Redo reducer. Text stays in the browser by default, and the project includes responsive/accessibility work plus automated audit tests."

## What to Demonstrate Live

18. Paste a messy multiline list with duplicate values and extra spaces.
19. Show live statistics updating immediately.
20. Use Remove Extra Spaces, then Remove Duplicate Lines.
21. Undo and Redo to prove history architecture.
22. Search for a literal special character such as `` to explain regex escaping.
23. Import a small text file; if text already exists, show Replace/Append/Cancel.
24. Switch light/dark mode and narrow the browser to demonstrate responsiveness.
25. Copy or download the processed text and mention that no server is used.

## Questions You Should Be Ready For

- Why React instead of plain JavaScript?
- Why no backend/database?
- Why use Set for duplicate lines?
- How do you handle emoji/Unicode?
- How is Regex mode made safe?
- How exactly does Undo/Redo work?

## Strong Portfolio Talking Points

| Avoid saying only...  | Say this instead...                                                                                                    |
|-----------------------|------------------------------------------------------------------------------------------------------------------------|
| "It converts text."   | "I centralized deterministic text algorithms in pure utilities and connected them to React state/history."             |
| "It supports Bangla." | "The code avoids ASCII-only operations using Intl.Segmenter, Unicode property escapes and Array.from where practical." |
| "It is secure."       | "There is no backend or external text API; source audit checks network paths, and draft persistence is opt-in."        |
| "It has undo."        | "A bounded reducer maintains current/undo/redo states and clears incompatible redo branches."                          |

# Appendix A — File-by-File Reference

A compact map you can revisit when reading the source code.

| File                         | Why it exists                                                               |
|------------------------------|-----------------------------------------------------------------------------|
| index.html                   | HTML shell with the React root element.                                     |
| src/main.jsx                 | Creates the React root, enables StrictMode and imports CSS.                 |
| src/App.jsx                  | Main coordinator: state, stats, tools, import/export, settings and dialogs. |
| components/Header.jsx        | Brand, theme, import/download/settings actions.                             |
| components/StatsGrid.jsx     | Displays primary live statistics.                                           |
| components/Editor.jsx        | Textarea, editor actions and drop handling.                                 |
| components/ToolPanel.jsx     | Tool categories, options and Find/Replace UI.                               |
| components/TextInsights.jsx  | Unique words and top-word frequency.                                        |
| components/ModalShell.jsx    | Accessible modal foundation with keyboard/focus behavior.                   |
| components/ConfirmDialog.jsx | Clear confirmation.                                                         |
| components/ImportDialog.jsx  | Replace/Append/Cancel import safety.                                        |
| components/SettingsModal.jsx | Editor settings and local draft controls.                                   |
| components/Toast.jsx         | Non-blocking feedback messages.                                             |
| utils/textUtils.js           | All core text-processing/statistics pure functions.                         |
| utils/historyUtils.js        | Pure bounded history reducer.                                               |
| utils/fileUtils.js           | File validation/read, clipboard and download helpers.                       |
| utils/storage.js             | Safe localStorage wrapper and keys.                                         |
| hooks/useTextHistory.js      | React API around history reducer.                                           |
| hooks/useDebounceValue.js    | 150 ms delayed value for heavier analysis.                                  |
| data/stopwords.js            | Local English stopword Set.                                                 |
| data/sampleText.js           | Demo text loaded only on explicit action.                                   |
| styles/globals.css           | Theme tokens and global resets.                                             |
| styles/app.css               | Main product styling.                                                       |
| styles/responsive.css        | Breakpoints and mobile/tablet reflow.                                       |
| tests/text-utils.test.mjs    | Behavior tests for algorithms and history.                                  |
| tests/project-audit.test.mjs | Project architecture/privacy/responsive audit.                              |
| README.md                    | Public GitHub-facing documentation.                                         |
| PROJECT_GUIDE.md             | Detailed student explanation + viva Q&A.                                    |
| VALIDATION.md                | Final audit status and build-environment caveat.                            |

# Appendix B — Viva Questions & Answers

25 concise questions based on the actual implementation.

## 1. What is the purpose of Text Utility Toolkit?

To combine common text analysis, cleaning, transformation and export tasks in one privacy-friendly browser workspace.

## 2. Why is React used?

The UI contains many synchronized states—editor content, statistics, tool options, dialogs, settings and history—so React provides a clean component/state model.

## 3. Why is Vite used?

It provides a fast development server and optimized production build with minimal React configuration.

## 4. Why does the project not need a database?

The app processes temporary local text and has no account/shared-record requirement.

## 5. Where does text processing happen?

Inside the browser using JavaScript utility functions.

## 6. How is word count calculated?

Intl.Segmenter is preferred; a Unicode-aware regex fallback is available.

## 7. What happens for empty text?

Core stats return zero values; word count is not incorrectly reported as one.

## 8. How are characters counted?

`Array.from(text).length` is used for code-point-aware counting.

## 9. How are lines counted?

Line endings are normalized to LF, then newline-separated lines are counted; empty text is 0.

## 10. Is sentence count exact?

No. It is a documented punctuation-based heuristic.

## 11. How is reading time estimated?

Word count divided by 200 words per minute.

## 12. How are duplicate lines removed?

A Set stores comparison keys while the first occurrence of each original line is kept.

## 13. Why use Set?

It provides efficient membership checks and naturally models uniqueness.

## 14. How is first occurrence order preserved?

Lines are scanned from top to bottom and only unseen keys are appended.

## 15. How does sorting work?

A copied array is sorted; alphabetic modes use `localeCompare()` with numeric comparison.

**16. Why not use `split("")` for reverse text?**

It can break Unicode surrogate pairs/emoji; the project prefers grapheme segmentation and falls back to `Array.from()`.

**17. How is normal Find mode made safe?**

Regex metacharacters are escaped so symbols such as `.` or `?` are treated literally.

**18. What happens with invalid Regex input?**

Errors are caught and shown as validation feedback instead of crashing the app.

**19. What is the Clipboard API used for?**

`navigator.clipboard.writeText()` copies the current editor text.

**20. What is Blob used for?**

A UTF-8 text Blob is converted to an object URL to trigger a local file download.

**21. How are text files imported?**

The browser gives a File object; the project validates it and reads it locally with `file.text()`.

**22. How does Undo/Redo work?**

A pure reducer stores current text plus bounded undo and redo arrays for explicit transformations.

**23. Why is the history bounded?**

Each text state can be large, so unlimited history would waste memory; the limit is 40 states.

**24. Is user text saved in `localStorage`?**

Not by default. Only theme/settings are persisted; draft text is stored only when the user enables auto-save.

**25. What are the main limitations?**

Heuristic sentences, language-dependent tokenization, browser-dependent clipboard/segmenter behavior, and possible performance cost for extremely large text.

## Final Takeaway

**If you remember only one thing**

This project is valuable because it turns simple string operations into a complete product: clean architecture, deterministic algorithms, browser APIs, privacy-first storage, Unicode awareness, recoverable transformations, responsive/accessible UI and testable behavior.